

Atividade Prática #2

Algoritmos de Busca no Jogo 8-Puzzle

Aluno: João Comini César de Andrade

1. Introdução

O **8-puzzle** é um problema clássico de Inteligência Artificial que consiste em um tabuleiro 3×3 com oito peças numeradas (1 a 8) e um espaço vazio. O objetivo é transformar uma configuração inicial qualquer na configuração final (estado objetivo) por meio de movimentos que deslizam a peça adjacente ao espaço vazio para cima, baixo, esquerda ou direita.

Este trabalho modela o 8-puzzle como um problema de busca em espaço de estados e implementa, em Python, cinco algoritmos distintos: BFS, DFS, Busca de Custo Uniforme (UCS), Busca Gulosa e A* com três heurísticas diferentes. Os algoritmos são comparados experimentalmente em termos de tempo de execução, nós visitados, nós gerados e a profundidade da solução.

2. Modelagem do Problema

Estado: Tupla de 9 inteiros representando o tabuleiro em ordem de leitura (linha × coluna). O valor 0 representa o espaço vazio. Ex.: (0,1,2,3,4,5,6,7,8) = estado objetivo.

Estado inicial: Qualquer permutação válida (solucionável) dos valores 0–8.

Estado objetivo: (0,1,2,3,4,5,6,7,8) espaço vazio no canto superior esquerdo.

Ações: Mover a peça adjacente ao espaço vazio para Cima, Baixo, Esquerda ou Direita.

Modelo de transição: Troca o espaço vazio com a peça na direção do movimento.

Teste do objetivo: Verificar se o estado corrente é igual ao estado objetivo.

Custo: Cada movimento tem custo 1 (custo uniforme).

Deteção de instâncias sem solução: Um estado é solucionável se e somente se o número de inversões (pares de peças fora de ordem) for par. O programa verifica isso antes de executar qualquer busca e informa o usuário caso o estado inicial seja insolucionável.

3. Descrição dos Algoritmos

3.1 BFS — Busca em Largura

Explora os nós em ordem crescente de profundidade utilizando uma fila (FIFO). É **completo** (encontra solução se existir) e **ótimo** quando todos os custos de passo são iguais. Complexidade de tempo e espaço: $O(b^d)$, onde b é o fator de ramificação e d a profundidade da solução. Para o 8-puzzle, $b \approx 2,67$ e d pode chegar a 31.

3.2 DFS — Busca em Profundidade

Explora um ramo da árvore de busca até o limite antes de retroceder, utilizando uma pilha (LIFO). **Não é completo** em espaços de estados cíclicos sem verificação de estados

visitados e **não é ótimo**. Complexidade de espaço: $O(b \cdot m)$, onde m é a profundidade máxima. Implementado com limite de profundidade (50) para evitar loops.

3.3 UCS — Busca de Custo Uniforme

Generalização do BFS que expande o nó de menor custo acumulado $g(n)$, usando uma fila de prioridade. É **completo** e **ótimo** para custos não negativos. Para o 8-puzzle com custo uniforme (todos os passos custam 1), é equivalente ao BFS.

3.4 Greedy — Busca Gulosa

Expande o nó com menor valor heurístico $h(n)$, sem considerar o custo acumulado. **Não é completo** nem **ótimo** em geral. Muito rápido na prática, mas pode encontrar soluções subótimas. Utiliza a heurística de Distância Manhattan nesta implementação.

3.5 A* — Busca Informada Ótima

Combina o custo acumulado $g(n)$ com a heurística $h(n)$: $f(n) = g(n) + h(n)$. É **completo** e **ótimo** quando a heurística é admissível (nunca superestima). É o algoritmo de referência para este tipo de problema. Implementado com três heurísticas distintas.

4. Heurísticas para o A*

H1 — Peças Fora do Lugar (Misplaced Tiles)

Conta o número de peças que não estão na posição objetivo (excluindo o espaço vazio). É **admissível** porque cada peça fora do lugar precisa de pelo menos 1 movimento para chegar ao objetivo. É uma heurística relativamente fraca pois ignora a distância real necessária.

H2 — Distância Manhattan

Soma das distâncias de Manhattan ($|\Delta_{row}| + |\Delta_{col}|$) de cada peça até sua posição objetivo. É **admissível** e **consistente**, pois nunca superestima (cada passo move uma peça apenas 1 posição) e domina H1. É a heurística padrão para o 8-puzzle.

H3 — Conflito Linear (Linear Conflict)

Extensão da Distância Manhattan que adiciona 2 para cada par de peças em conflito linear: duas peças na mesma linha (ou coluna) cujas posições objetivo estão também na mesma linha (coluna) mas em ordem invertida. É **admissível** e domina H2 (nunca subestima mais). Reduz significativamente o número de nós expandidos.

5. Resultados Experimentais

Foram testadas três instâncias de dificuldade crescente. Os resultados abaixo mostram a profundidade da solução ótima, o número de nós visitados (expandidos), o número de nós gerados e o tempo de execução.

Instância 1 — Fácil (3 movimentos)

Estado: 1 2 5 | 3 4 0 | 6 7 8

Algoritmo	Prof.	Visitados	Gerados	Tempo (ms)
BFS	3	11	14	0.18
DFS	3	127090	242791	2540.32
UCS	3	11	31	0.44
Greedy (Manhattan)	3	4	9	0.12
A* Misplaced	3	4	9	0.10
A* Manhattan	3	4	9	0.18
A* Linear Conflict	3	4	9	0.15

Instância 2 — Média (3 movimentos)

Estado: 1 4 2 | 3 7 5 | 6 0 8

Algoritmo	Prof.	Visitados	Gerados	Tempo (ms)
BFS	3	10	14	0.26
DFS	3	9	11	0.16
UCS	3	10	27	0.35
Greedy (Manhattan)	3	4	11	0.20
A* Misplaced	3	4	11	0.19
A* Manhattan	3	4	11	0.21
A* Linear Conflict	3	4	11	0.32

Instância 3 — Difícil (28 movimentos)

Estado: 8 7 6 | 5 4 3 | 2 1 0

Algoritmo	Prof.	Visitados	Gerados	Tempo (ms)
BFS	28	178224	460298	4547.33

Algoritmo	Prof.	Visitados	Gerados	Tempo (ms)
DFS	50	51369	87505	795.66
UCS	28	178224	476081	4969.45
Greedy (Manhattan)	28	35	86	1.16
A* Misplaced	28	68754	184803	1980.17
A* Manhattan	28	727	1833	19.47
A* Linear Conflict	28	420	1092	16.41

6. Análise Comparativa

a) Qual algoritmo foi mais rápido? Qual visitou menos nós?

O A* com Conflito Linear foi o mais eficiente na instância difícil, visitando apenas 420 nós em 16,4 ms. O A* Manhattan ficou próximo (727 nós, 19,5 ms). A Busca Gulosa visitou apenas 35 nós, mas não garante otimalidade. BFS e UCS foram as piores, visitando mais de 178.000 nós e demorando mais de 4 segundos. O DFS foi ineficiente na instância fácil (127.090 nós) pela sua natureza de explorar ramos longos.

b) Como as três heurísticas impactaram o desempenho do A*?

A heurística mais forte (Conflito Linear) reduziu o número de nós visitados em 42% em relação à Manhattan e 99,4% em relação à Misplaced na instância difícil. A ordem de dominância observada foi: Linear Conflict $\geq$ Manhattan $\geq$ Misplaced, conforme esperado pela teoria. Heurísticas mais informadas permitem maior poda do espaço de busca.

c) Houve diferença nos caminhos encontrados entre os algoritmos?

Todos os algoritmos ótimos (BFS, UCS, A* com heurísticas admissíveis) encontraram a solução de profundidade mínima (28 na instância difícil). O DFS encontrou uma solução de profundidade 50 (subótima). A Busca Gulosa encontrou profundidade 28 neste caso, mas isso não é garantido em geral.

d) Em quais situações recomendar cada algoritmo?

- **A*** (Manhattan ou Conflito Linear): Situações gerais onde se deseja solução ótima com eficiência. Melhor custo-benefício para o 8-puzzle.
 - **BFS / UCS**: Problemas com profundidade pequena ou quando se quer garantia de otimalidade sem heurística.
 - **DFS**: Problemas com soluções em ramos profundos e restrições severas de memória.
 - **Greedy**: Quando velocidade é mais importante que otimalidade e há uma boa heurística disponível.
-

7. Diário de Decisões

a) Maior dificuldade técnica e como foi resolvida

A maior dificuldade foi garantir que o DFS não entrasse em loops infinitos no espaço de estados do 8-puzzle, que é cíclico (ex.: mover direita e depois esquerda retorna ao estado original). Isso foi resolvido mantendo um conjunto de estados visitados e adicionando um limite de profundidade (50) para evitar buscas excessivamente longas.

b) O que foi tentado e não funcionou

Inicialmente, a heurística de Conflito Linear foi implementada de forma incorreta, contando conflitos duplicados (o par (A,B) era contado duas vezes). Isso tornava a heurística inadmissível. O problema foi identificado ao comparar resultados com a Manhattan e o A* com Conflito Linear estava retornando soluções subótimas. A correção foi verificar cada par (i,j) com $i < j$ apenas uma vez.

c) Melhorias futuras

Com mais tempo, seria interessante implementar: (1) IDA* (Iterative Deepening A*), que combina a eficiência de memória do DFS com a otimalidade do A*; (2) uma interface gráfica visual para acompanhar o caminho passo a passo; (3) geração aleatória de instâncias com profundidade controlada para benchmarks mais sistemáticos; (4) a heurística de banco de padrões (Pattern Database), considerada a mais eficiente para o 8-puzzle.

8. Instruções de Execução

Requisitos: Python 3.8 ou superior (sem dependências externas).

Executar o programa interativo:

```
python3 puzzle8.py
```

Exemplos de entrada (9 números, 0 = espaço vazio):

- 0 1 2 3 4 5 6 7 8 → estado inicial igual ao objetivo
- 1 2 5 3 4 0 6 7 8 → instância fácil (3 movimentos)
- 8 7 6 5 4 3 2 1 0 → instância difícil (28 movimentos)

O programa solicita o estado inicial, verifica se é solucionável e apresenta menu para selecionar o algoritmo. Ao final, exibe o caminho completo passo a passo, o número de nós visitados, a profundidade da solução e o tempo de execução.